TZ受信機 性能評価スクリプト 使い方マニュアル

版 0.3（2026-10-21）／ 実機での動作は未確認

1. このマニュアルについて

TZ受信機の性能を測るスクリプト群の使い方を説明する。測るのは主に次の2つで、観測データの解析には使わない。

量	意味
受信機雑音温度 Trx	受信機そのものが出す雑音の大きさを温度で表したもの。低いほど良い
イメージ抑圧比 IRR	TZ は2つのサイドバンド（USB と LSB）を分けて出力する受信機（2SB 受信機）である。片方のサイドバンドの出力に、もう片方の信号がどれだけ漏れずに済んでいるかの比。大きいほど良い

これらは、SISミクサにかけるバイアス電圧と、LO（局部発振信号）の強さを決める LO減衰器の電流によって変わる。 そのため、この2つを振りながら Trx と IRR を測り、良い組み合わせ（運用値）を決める。

スクリプトは、以前 ROS 上で動いていたもの（旧システム）を、ROS を使わない現在の制御ソフト（tz_controller）の上に移植したものである。 測定の手順と解析の計算は旧システムと同じにしてあり、旧システムの結果とそのまま比べられる。

2. 用語

用語	意味
旧システム	ROS で動いていた制御一式（2019-2025）。旧スクリプトは制御PCで動いていた
新システム	tz_controller と TZ-tools。ROS なしで FAC 上で動く
制御PC	TZ の運用で VNC から入る PC。1st LO の信号発生器（Anapico SG）はここから操作する
FAC	DA/AD ボードが刺さっていて、SISバイアスを直接扱う PC。このスクリプト群は FAC の ~/tz_workspace の下の専用フォルダに置き、そのフォルダで動かす
素子名	SISミクサ4本の名前 v1, h1, h2, v2。v/h は偏波、1/2 は1つの偏波を受ける2本のミクサを表す

IF系統	受信機の出力4本。HU, HL, VU, VL（偏波 H/V とサイドバンド U/L の組み合わせ）。IFスイッチでどれか1本をスペアナに送る
HOT と COLD	受信機に見せる2つの基準の温度。HOT は室温の吸収体（300 K とする）、COLD は液体窒素に浸した吸収体（77 K とする）
Y-factor 法	HOT を見たときと COLD を見たときの出力の比 Y から Trx を求める方法。Trx = (300 − 77 Y)/(Y − 1)
チョッパー	HOT と COLD を一定の周期で交互に見せる回転板（チョッパーホイール）。スペアナのゼロスパン表示に、HOT と COLD の出力が交互に並んだ波形が出る
スライダー	HOT と COLD を手で切り替える方式。Old MMC スイッチで Hot/Cold を選ぶ
Bias1, Bias2	Trx マップで振る2つのバイアス。Bias1 は v1 と h1、Bias2 は v2 と h2 に同じ値を入れる（両偏波を同時に動かす）

3. スクリプトの一覧

新スクリプト	何をするか	HOT/COLD	旧スクリプト
trx_map.py	Bias1 と Bias2 を格子状に振り、1つの IF系統の Trx マップを作る	チョッパー	yfactor_with_search_sis_vol_direction-ver2.py、同 -ver2_auto.py
loatt_search.py	LO減衰器の電流を振り、HL と VL の Trx を測る	チョッパー	yfactor_with_search_loatt_level_direction-ver2.py
irr_measure.py	1組のバイアスで、両偏波の IRR と Trx を測る	スライダー	irr_measurement2.py
yfactor_chopper.py	4つの IF系統の Trx を測る	チョッパー	yfactor_chopper_direction.py
yfactor_hotcold.py	4つの IF系統の Trx と、その周波数特性を測る	スライダー	yfactor_sa_direction.py
set_bias.py	4本の SISバイアスを設定する。`--tuned` で運用値	なし	set_sis_vol_direction.py
set_loatt.py	LO減衰器の電流（H, V）を設定する	なし	set_loatt_level_direction-ver2.py
pol_test.py	DA端子を1本ずつ 0 mV にして、どの IF系統の出力が変わるかを見る。素子名と偏波の対応を決めるため	なし（LO は必要）	なし（新規）

settle_test.py	バイアスを変えてから Trx が落ち着くまでの時間を測る（trx_map の待ち時間を決めるため）	チョッパー	なし（新規）
timing_test.py	バイアスの追従、スペアナの読み出しと掃引、IFスイッチの切り替わりにかかる時間を測る。LO も液体窒素も要らない	なし	なし（新規）
iv_curve_measure.py	4本のミクサの IV カーブを測る。`--ros` で旧システムの換算	なし	sis_iv_curve_direction.py
rx_performance_analysis.py	上の測定結果から Trx、IRR、マップを計算する（測定の最後に自動で動く）	なし	解析用の Jupyter notebook

運用値を決めるときの流れ

1. TZ をチューニングする（立ち上げ手順。LO減衰器はそのときの値）
2. **trx_map.py** で、IF系統ごとに SISバイアスの Trx マップを作り、Trx の低い Bias1, Bias2 を選ぶ
3. 選んだバイアスで **loatt_search.py** を走らせ、Trx の低い LO減衰器の電流を選ぶ
4. 選んだバイアスの付近で、バイアスを少しずつ変えて **irr_measure.py** を何回か走らせ、**irr-map** で IRR のマップを作る
5. IRR が 10 dB を超え、かつ Trx が低いバイアスを運用値とする
6. 運用値でチューニングし直し、**yfactor_chopper.py** と **yfactor_hotcold.py** で最終的な Trx を測る

4. 測定の準備

光学系とスペアナの準備は、TZマニュアル（立ち上げ/シャットダウン/性能評価測定編）の「性能評価測定前準備と測定の流れ」に従う。要点を下にまとめる。

4.1 共通

- Trx を求める測定では液体窒素を使う。液体窒素の入った容器を Cold Port に置き、容器内の吸収体が全部つかるまで入れる
- 容器を覆うビニールに水滴がつくと COLD の温度が狂う。測定中もときどき確認する（10月以降はつかないことが多い）

4.2 チョッパーを使う測定（trx_map, loatt_search, yfactor_chopper）

1. 観測棟で LD1 の Hot を入れる（ミラー#4 手前の吸収体）

2. 第四反射鏡を OLD にする
3. Old MMC スイッチを Rotate と Cold にする
4. Old Beam Mirror 切り替えパネル: #7 Total Reflection、#8 86/40G Separation（filter）または House（through）、#9 どれでもよい
5. TZ をチューニングする
6. スペアナでゼロスパン表示に HOT と COLD が交互に並んだ波形が出ていることを確かめる。Att. や Ref. は見やすいよう適宜変える

4.3 スライダーを使う測定（yfactor_hotcold）

1. Old MMC スイッチを R にする
2. Old Beam Mirror 切り替えパネルはチョッパーと同じ
3. TZ をチューニングする
4. スクリプトが HOT/COLD を求めてきたら、Old MMC を Hot/Cold に切り替える

4.4 IRR 測定（irr_measure）

1. Old MMC スイッチを R にする
2. Old Beam Mirror 切り替えパネル: #7 Total Reflection、#8 House（through）、#9 どれでもよい
3. TZ をチューニングする
4. 試験信号（Phase Matrix SG）が出てスペアナに見えることを、最初に一度確かめる
5. スクリプトが HOT/COLD を求めてきたら、Old MMC を Hot/Cold に切り替える

4.5 スペアナの設定

スクリプトが自動で次の設定にする（TZマニュアルと同じ）。Att. と Ref. は変えないので、必要なら事前に手で合わせる。

測定	Center	Span	RBW	VBW	Sweep	平均
チョッパー	6 GHz	0 Hz	1 MHz	300 Hz	300 ms	なし
スライダー	6 GHz	4 GHz	1 MHz	1 MHz	自動	20回
IRR	6 GHz	500 kHz	300 Hz	10 Hz	自動	5回

平均をとる測定では、IF系統や HOT/COLD を切り替えるたびに平均をやり直させ、決められた回数の掃引が終わってから記録する。 切り替え前のデータが平均に混ざらないようにするためである（旧システムでは、平均の完了を人が目で確かめていた）。 irr_measure.py では、スペアナの平均の代わりに PC 側で平均する方法も選べる（6.3）。

5. 共通の使い方

5.1 実行の形

FAC に入り、スクリプトを置いたフォルダ（~/tz_workspace の下）に移って、スクリプト名の後に引数を並べて実行する。結果のフォルダもそこにできる。

```
cd ~/tz_workspace/（スクリプトのフォルダ）
uv run python trx_map.py HL --bias1 6.0 8.0 21 --bias2 6.0 8.0 21
```

どのスクリプトも -h を付けると引数の一覧が出る。旧スクリプトで対話入力していた値の多くは、引数で渡す形にした。 人の操作（HOT/COLD の切り替えなど）が必要なところでは、旧スクリプトと同じく画面に指示が出て、y の入力を待つ。

5.2 共通オプション

オプション	意味
--settle 秒	IF系統を切り替えた後の待ち時間。既定値は旧システムと同じ（多くは 10 秒）。切り替え直後に出力が落ち着かない場合は長くする
--keep-sa	スペアナの設定を変えない（手で設定したまま測る）。このとき平均はかけない設定にしておくこと
--dry-run	疑似機器で動かす。受信機には何も送らない（8章）
--yes	確認の [y/n] にすべて y と答える。--dry-run の試運転用で、本番では使わない

5.3 出力

実行した場所に、測定1回ごとにフォルダができる。

```
trx_map_261021_143005/
  meta.json      測定条件（引数、素子名とDA端子の対応、換算係数、スペアナ設定など）
  traces.csv     1行がスペアナの1回分の表示データ。先頭の列が条件（IF系統、バイアスなど）、
                 続く p000, p001, ... がスペアナの各点の値（dBm）
  result.csv     解析結果（測定の最後に自動で作られる）
  *.png          図
```

CSV なので、Excel や Python（pandas）でそのまま開ける。

5.4 安全のための確認

- 起動時に、設定ファイルのバイアスの換算係数が想定どおりかを確かめ、外れていれば何も出力せずに止まる
- どのバイアスも 0 から 9.0 mV の外なら、出力する前に止まる（範囲は設定ファイルで変えられる）
- SISミクサに負担をかけないよう、範囲や点数は推奨値から大きく外さない

6. 各スクリプト

6.1 trx_map.py：SISバイアスの Trx マップ

```
uv run python trx_map.py {HU,HL,VU,VL} --bias1 START STOP NUM --bias2 START STOP NUM
[--auto]
```

引数	意味	推奨
HU など	測る IF系統	4系統それぞれ1回ずつ
`--bias1 START STOP NUM`	v1 と h1 に同じ値を入れて振る。START から STOP まで NUM 点 [mV]	6 8 21
`--bias2 START STOP NUM`	v2 と h2 に同じ値を入れて振る	6 8 21
`--auto`	Bias1 を1段進めるたびの確認（y の入力）をしない	ビニールに水滴がつかない時期なら付けると効率的

流れ: Bias1 の値ごとに v1 と h1 を設定する。`--auto` がなければここで y を待つ（液体窒素の容器の様子を確かめる機会）。続いて Bias2 の値ごとに v2 と h2 を設定し、5.8 秒待ってから記録する。

所要時間: 21 × 21 点で約45分。起動時に目安を表示する。

結果: trx_map.png。各マスの色と数字は同じ値から描いている。Trx が低く、周りも低い領域の中心を選ぶ。

> 終了時に4本のバイアスをすべて 0 にする（旧システムと同じ）。測定後は運用値でチューニングし直すこと。

6.2 loatt_search.py：LO減衰器の電流と Trx

```
uv run python loatt_search.py START STOP NUM FREQ [--bias V1 H1 H2 V2]
```

引数	意味	推奨

START STOP NUM	LO減衰器の電流 [mA] を START から STOP まで NUM 点	16 30 15
FREQ	1st LO SG の周波数 [GHz]（記録用）	15.609
`--bias`	測定中に保つバイアス [mV]。省くと設定ファイルの運用値	trx_map で選んだ値

流れ: バイアスを設定し y を待つ。電流の値ごとに H 側を設定して 60 秒待ち、V 側を設定する。続いて HL と VL を記録する。最後に、残す H と V の電流を聞かれるので入力すると、その値に設定して終わる。

結果: trx_vs_loatt.png。Trx が最も低くなる電流を選ぶ。

6.3 irr_measure.py：IRR

```
uv run python irr_measure.py V1 H1 H2 V2 [--usb-freq 16.609] [--lsb-freq 14.609] [--soft-average N]
```

引数	意味
V1 H1 H2 V2	バイアス [mV]。この順に並べる
`--usb-freq`, `--lsb-freq`	試験信号 SG の周波数 [GHz]。SG の出力は6逓倍されて受信機に入る。USB は 16.609（RF 99.654 GHz）、LSB は 14.609（RF 87.654 GHz）。省くと聞かれる
`--soft-average N`	スペアナの平均を切り、掃引 N 回分を1回ずつ記録して PC 側で平均する。切り替え後に始まった掃引だけを使うので前の系統が混ざらず、掃引どうしのばらつきから IRR と Trx の誤差（1σ）も出る。測定時間はスペアナで N 回平均するのと同じ。省くとスペアナの平均を使う

流れ: バイアスを設定し、試験信号を止める。COLD、HOT の順に、Old MMC の切り替えを指示されるので従う。それぞれで4つの IF系統を記録する。続いて USB、LSB の試験信号を順に出し、同じく4系統を記録する。各系統で、切り替え、待ち、y の入力、記録の順に進む。

結果: 偏波ごとの IRR（USB, LSB）と Trx。バイアスは設定したまま終わる。バイアスを変えて何回か測ったら、次のようにまとめてマップにする。

```
python rx_performance_analysis.py irr-map irr_map_out irr_measure_*
```

6.4 yfactor_chopper.py：4系統の Trx（チョッパー）

```
uv run python yfactor_chopper.py {filter,nofilter} FREQ
```

filter/nofilter は周波数分離フィルタの有無、FREQ は 1st LO SG の周波数（どちらも記録用）。バイアスと LO は事前に設定しておく（このスクリプトは変えない）。4つの IF系統を順に切り替え、10 秒待って記録する。

6.5 yfactor_hotcold.py：4系統の Trx と周波数特性（スライダー）

```
uv run python yfactor_hotcold.py {filter,nofilter} [--freq FREQ]
```

HOT にして y、4系統を記録、COLD にして y、4系統を記録、の順に進む。各系統で平均の完了を待ってから記録する。 結果は系統ごとの Trx（result.csv）と、IF 周波数ごとの Trx（trx_spectrum.csv, trx_spectrum.png）。

6.6 settle_test.py：バイアスを変えた後の落ち着き

```
uv run python settle_test.py HU --a 6.4 7.3 7.3 6.3 --b 6.4 7.3 7.8 6.3 [--repeat 5]
[--watch 10]
```

Trx の違う2点 A と B のあいだでバイアスを行き来させ、変えるたびに 10 秒間トレースを続けて読む。A には今のバイアスを入れる（最後に A に戻る）。 結果は、変えてからの時間と Trx の図（settle_test.png）と、何秒後から最終値の 5% 以内に収まったかの表。チョッパーと液体窒素が要る。

6.7 timing_test.py：待ち時間の試験（LO、液体窒素なし）

```
uv run python timing_test.py [--only bias,sa,if,soft]
```

試験	分かること
bias	各ミクサのバイアスを 2 mV と 3 mV のあいだで動かし、電圧モニタが新しい値に落ち着くまでの時間
sa	チョッパーと IRR の設定それぞれで、掃引時間、トレース1本の読み出し時間、トレースが新しくなる間隔
if	IF系統を切り替えてから、新しい系統の出力がトレースに現れるまでの時間
soft	PC 側の平均（irr_measure の `--soft-average`）が、実機で掃引を1回ずつ正しく拾えるか

結果は timing_test_（日時）フォルダの summary.txt にまとまる。終わるとバイアスはすべて 0 になる。

6.8 iv_curve_measure.py：IV カーブ

```
uv run python iv_curve_measure.py          （2026-09 と同じ換算）
uv run python iv_curve_measure.py --ros    （旧システムの換算）
```

4本のミクサを1本ずつ −5 から 10 mV まで 0.1 mV 刻みで掃引し、電圧と電流を読む。開始前に条件が表示され、yes と打つと始まる。終わるとバイアスは 0 になる。 2026-09 の設定は、旧システムと比べて DA の換算が 12% 低く、AD の読みが2倍になる。比べるときは `--ros` の結果を使う。

6.9 set_bias.py, set_loatt.py：設定だけを行う

```
uv run python set_bias.py 6.4 7.3 7.3 6.3      （V1 H1 H2 V2 [mV]。--tuned で運用値）
uv run python set_loatt.py 23 23               （H V [mA]）
```

値は終了後もそのまま残る。素子名、DA端子、換算は device_config_ros.toml に従う。

6.10 pol_test.py：素子名と偏波の対応

```
uv run python pol_test.py
```

LO を入れた状態で、運用値のバイアスから DA端子を1本ずつ 0 mV にし、4つの IF系統の出力の変化（dB）を表にする。HU と HL が変われば H偏波、VU と VL が変われば V偏波のミクサである。終わるとバイアスは運用値に戻る。所要は約3分。

7. 解析（rx_performance_analysis.py）

測定スクリプトは最後に自動で解析する。解析だけやり直すときは、測定のフォルダを指定して実行する。

```
python rx_performance_analysis.py trx_map_261021_143005
```

計算	方法
チョッパーの Trx	ゼロスパンの波形から HOT と COLD の切り替わり目を除き、平均より上を HOT、下を COLD の出力とする
スライダーの Trx	HOT と COLD の出力の差（dB）を平均して Y を求める。周波数特性は各周波数の点ごとに求める
Y から Trx	Trx = (300 − 77 Y)/(Y − 1)
IRR	HOT と COLD の出力と、試験信号の強さ（各 IF系統で 信号 − HOT が最大の点）から計算する

計算は旧システムの解析と同じで、旧システムの測定データで結果が一致することを確かめてある。

> 旧システムで作られた数字入りのマップには誤りがあった。Trx マップは数字の行と列が入れ替わっていた（色が正しい）。IRR マップは色が1列ずれることがあった（数字が正しい）。旧システムの図を見返すときは注意する。

8. 疑似機器での試運転（`--dry-run`）

受信機につながずに、スクリプトが最後まで動くかを確かめる機能。DAボード、スペアナ、IFスイッチ、LO減衰器、試験信号 SG の代わりに、それらしい形のデータを返す疑似機器を使う。

- 受信機には何も送らないので、手元の PC でも安全に動かせる
- 出てくる Trx や IRR の数値に意味はない。確かめられるのは、手順、記録、解析、作図が止まらずに動くこと
- 引数の打ち間違いや所要時間を、本番の前に確かめるのに使う

```
python trx_map.py HL --bias1 6 8 5 --bias2 6 8 5 --dry-run --yes
```

9. 設定ファイル（device_config_ros.toml）

旧システムの動作を再現するための設定。運用中の ~/tz_workspace/device_config.toml とは別のファイルで、このスクリプト群はそちらを読まない。

項目	値
バイアスの換算	指令値 [mV] ÷ (3 × 0.94) = DA出力 [V]。旧システムと同じで、ミクサにかかる電圧が指令値と1%以内で一致する
素子名とDA端子	v1 = DA ch7、h1 = DA ch6、h2 = DA ch5、v2 = DA ch8（旧システムが実際に出力していた対応）
運用値（tuned）	v1 6.4、h1 7.3、h2 7.3、v2 6.3 mV（2025-2026 シーズン）
LO減衰器	GPIB ポート4 が H、ポート5 が V。運用値はどちらも 23 mA
IFスイッチ	HU 105、HL 106、VU 107、VL 108
HOT, COLD の温度	300 K、77 K

> 運用中の device_config.toml とは、素子名とDA端子の対応、およびバイアスの換算が異なる。同じ「v1 = 6.4」でも、どちらの設定で動かすかで、かかる電圧とミクサが変わる。この食い違いは現在整理中である（10章）。

10. 注意点と未確定事項

- **実機では未試験。**初回は `--dry-run` で手順を確かめ、次に短い格子（例: 5 × 5 点）で試す
- **素子名と物理的な偏波の対応が未確定。**DA ch5 と ch7 のどちらが V偏波のミクサかが決まっていない。決まり次第、設定ファイルとこのマニュアルを直す
- **IFスイッチの動作。**open を送ると約3秒後にその系統に切り替わり、close では変わらないことを、旧システムの記録で確かめた。新スクリプトも旧システムと同じ順で送る
- **待ち時間。**バイアスや IF系統を変えた後の待ち時間は、旧システムと同じ値にしてある。旧システムの待ちはスペアナの更新の遅さで決まっていたので、timing_test と settle_test の結果で見直す予定

11. 改訂履歴

版	日付	内容
0.1	2026-10-21	初版
0.2	2026-10-21	TZ を知らない読者向けに全面改稿。測定準備、運用値を決める流れ、スペアナ設定（TZマニュアルに準拠）、平均の完了待ち、Trx の周波数特性を追加
0.3	2026-10-21	settle_test、timing_test、iv_curve_measure、set_bias、set_loatt、pol_test と、irr_measure の PC 側平均を追加。置き場所を専用フォルダに。IFスイッチの動作を確定

原稿は TZ-tools の docs/manual/rx_performance_manual.html。コードを変えたらここを直し、docs/manual/build_pdf.sh で PDF を作り直す。